



FINANCIAL FORECASTING OF ADULTS INCOME USING MACHINE LEARNING

Project Hand-out, Faculty Development Program

Team IDSWTID1749710097

Team Lead: Mayank Verma

Team Members:-

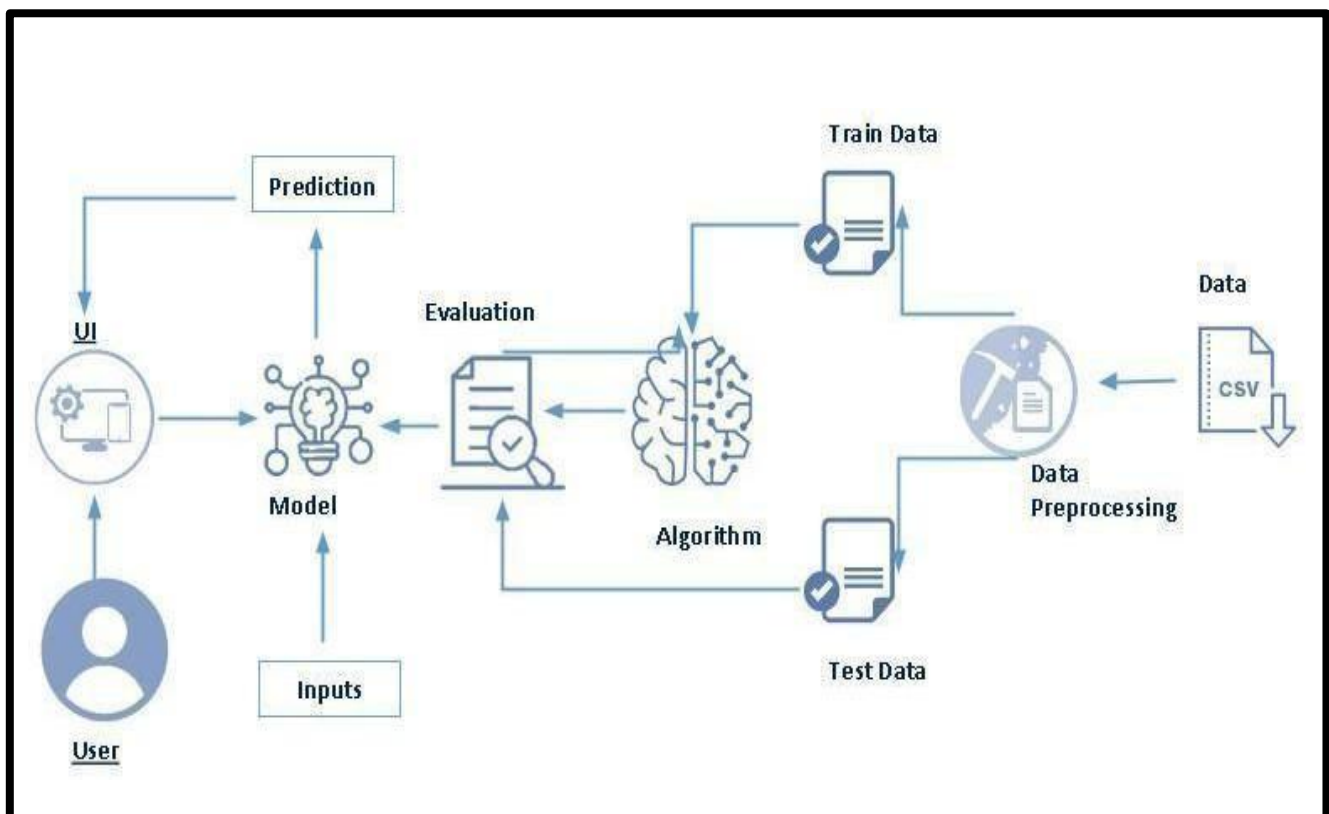
1. Sanjoli Gupta
2. Manas Pipersaniya
3. Sanjog Gupta

Financial Forecasting on Adult's Income Prediction Models

Financial forecasting is important as it aids businesses, investors, and analysts in predicting future events like stock prices, sales, revenues, and market trends. Conventional forecasting techniques tend to use hard assumptions and linear models, which could prove ineffective when dealing with a lot of intricate or tumultuous financial data. Machine Learning (ML) introduces a contemporary, data-centric methodology in financial prediction. By discovering patterns in past data, ML algorithms are capable of making more precise and dynamic predictions. Methods such as Linear Regression, Random Forest, Support Vector Machines, and deep learning models such as LSTM are employed to forecast time series data and identify trends, seasonality, and outliers.

This process enables organizations to act in response to changing markets, mitigate risk, and make better financial decisions. With the capacity to process enormous amounts of data and identify non-obvious relationships, machine learning is revolutionizing the way that we analyze and forecast financial results..

Technical Architecture:



Project Flow:

- User interacts with the UI to enter the input.
- Entered input is analysed by the model which is integrated.
- Once model analyses the input the prediction is showcased on the UI

To accomplish this, we have to complete all the activities listed below,

- Define Problem / Problem Understanding
 - Specify the business problem
 - Business requirements
 - Literature Survey
 - Social or Business Impact.
- Data Collection & Preparation
 - Collect the dataset
 - Data Preparation
- Exploratory Data Analysis
 - Descriptive statistical
 - Visual Analysis
- Model Building
 - Training the model in multiple algorithms
 - Testing the model
- Performance Testing & Hyperparameter Tuning
 - Testing model with multiple evaluation metrics
 - Comparing model accuracy before & after applying hyperparameter tuning
- Model Deployment
 - Save the best model
 - Integrate with Web Framework
- Project Demonstration & Documentation
 - Record explanation Video for project end to end solution
 - Project Documentation-Step by step project development procedure

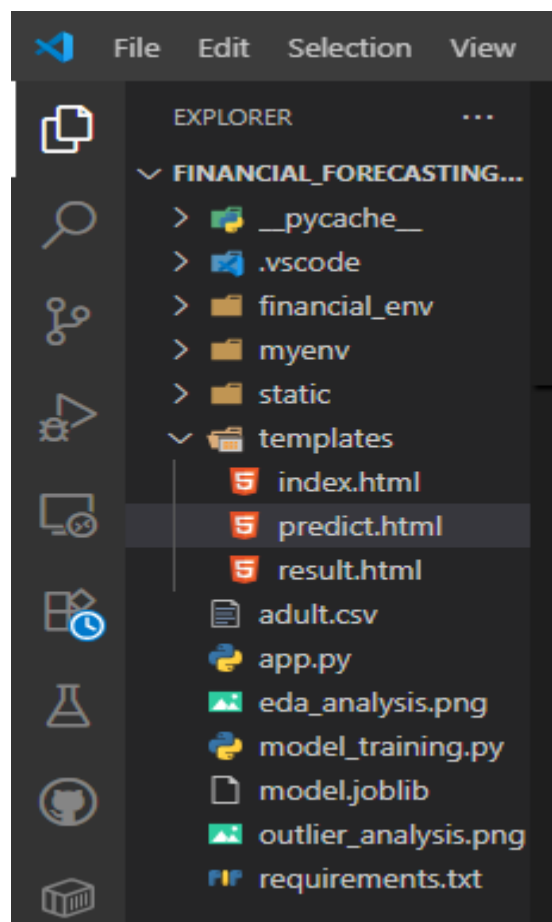
Prior Knowledge:

We must have prior knowledge of following topics to complete this project.

- **ML Concepts**
 1. Supervised learning: <https://www.javatpoint.com/supervised-machine-learning>
 2. Unsupervised learning: <https://www.javatpoint.com/unsupervised-machine-learning>
- **Decision tree:** <https://www.javatpoint.com/machine-learning-decision-tree-classification-algorithm>
- **Random forest:** <https://www.javatpoint.com/machine-learning-random-forest-algorithm>
- **KNN:** <https://www.javatpoint.com/k-nearest-neighbor-algorithm-for-machine-learning>
- **Xgboost:** <https://www.analyticsvidhya.com/blog/2018/09/an-end-to-end-guide-to-understand-the-math-behind-xgboost/>
- **Evaluation metrics:** <https://www.analyticsvidhya.com/blog/2019/08/11-important-model-evaluation-error-metrics/>
 - Flask Basics : https://www.youtube.com/watch?v=lj4l_CvBnt0

Project Structure:

Create the Project folder which contains files as shown below.



Create the Project folder which contains files as shown below.

- We are building a flask application which needs HTML pages stored in the templates folder and python script app.py for scripting.
- Model.joblib is our stored model. Later we will utilize this model in flask integration.
- Data Folder holds the Dataset utilized
- The Notebook file includes procedure for constructing the model.

Milestone 1: Define Problem / Problem Understanding

Activity 1: Specify the business problem

Refer Project Description

Activity 2: Business requirements

A financial forecasting project focused on predicting adult income can have various business requirements, depending on the specific goals and stakeholders involved. Some key requirements may include: -

1. **Data Accuracy:** Utilize accurate and current demographic and economic data for accurate projections.
2. **Scalability:** Process large dataset quickly, but also able to grow over time.
3. **Compliance:** Comply with privacy regulations (GDPR, CCPA) and guarantee ethical and fair predictions.
4. **Interpretability:** Understandable descriptions of predictions to help make decisions.
5. **Integration:** Integrate with finance, HR or policy-planning applications.
6. **Bias Mitigation:** Identify and mitigate biases with respect to gender, race, or negligence.
7. **Intuitive Outputs:** Provide actionable insights in easy-to-understand dashboards to a variety of users.
8. **Cost-effective:** Play the trade-off between accuracy and computational expensive.
9. **Ongoing Updates:** Regularly retrain models for changes in the economy and labor market.

Activity 3: Literature Survey

Recent literature highlights the growing application of machine learning (ML) in predicting adult income, with the objective of enhancing decision-making in both public and private sectors.

The canonical dataset often referenced is the UCI Adult Income dataset, originally extracted from U.S. Census data. It also acts as a reference for income classification research in association with socio-demographic variables such as age, educational level, marital status, and occupation.

Early works already proved that decision trees and Naive Bayes (see Kohavi, 1996) meet this requirement, and current works shows that better accuracy can be achieved by using ensemble methods such as Random Forest, Gradient Boosting, and neural networks. These models have been praised for modelling non-linear and interaction effects of the features. Recent works are also focused on fairness and bias mitigation, where various approaches have been proposed to address the ethical challenges including adversarial debiasing and algorithmic transparency.

In conclusion, the literature highlights the transition from rule-based and statistical methods to more refined data-driven methods, presenting ML as a robust, albeit delicate tool for socioeconomic forecasting.

Activity 4: Social or Business Impact

Social Impact:

Machine learning–driven financial forecasting for adult income can play a transformative role in promoting social equity and inclusivity. By identifying trends in income disparities across demographics, education levels, and regions, ML models enable policymakers to design targeted welfare programs and educational initiatives. This data-driven clarity also empowers individuals to make informed career and education choices, especially in underserved communities. However, it's crucial to ensure that models are fair and transparent to avoid perpetuating existing biases, especially when dealing with sensitive attributes like race, gender, and age.

Business Impact:

For businesses, financial forecasting using ML enhances strategic decision-making, especially in sectors like banking, insurance, real estate, and e-commerce. Companies can predict consumer spending power, tailor products for different income groups, and assess credit risk more accurately. These insights reduce financial uncertainty and optimize marketing and investment strategies. Moreover, it enables more personalized services, boosting customer satisfaction and loyalty.

Milestone 2: Data Collection & Preparation

ML depends heavily on data. It is the most crucial aspect that makes algorithm training possible. So, this section allows you to download the required dataset.

Activity 1: Collect the dataset

There are many popular open sources for collecting the data. Eg: kaggle.com, UCI repository, etc.

In this project we have used .csv data. This data is downloaded from kaggle.com. Please refer to the link given below to download the dataset.

Link: <https://www.kaggle.com/datasets/lovishbansal123/adult-census-income/data>

As the dataset is downloaded. Let us read and understand the data properly with the help of some visualization techniques and some analysing techniques.

Note: There are a number of techniques for understanding the data. But here we have used some of it. In an additional way, you can use multiple techniques.

Activity 1.1: Importing the libraries

Import the necessary libraries as shown in the image.

```
2 # model_training.py
3 import pandas as pd
4 import numpy as np
5 from sklearn.preprocessing import LabelEncoder
6 import matplotlib.pyplot as plt
7 import seaborn as sns
8 from sklearn.model_selection import train_test_split, cross_val_score, GridSearchCV
9 from sklearn.linear_model import LogisticRegression
10 from sklearn.tree import DecisionTreeClassifier
11 from sklearn.ensemble import RandomForestClassifier, AdaBoostClassifier
12 from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix
13 from imblearn.over_sampling import SMOTE
14 import joblib
15 import warnings
16 warnings.filterwarnings('ignore')
17
```

Activity 1.2: Read the Dataset

Our dataset is in .csv format which we can read the dataset with the help of pandas.

In pandas we have a function called `read_csv()` to read the dataset. As a parameter we have to give the directory of the csv file.

```
# Activity 1.1: Load and Explore Dataset
print("Loading Adult Census Income Dataset...")
df = pd.read_csv('adult.csv')

print(f"Dataset Shape: {df.shape}")
print("\nDataset Info:")
print(df.info())
```


- For checking the null values, `df.isna().any()` function is used. To sum those null values we use `.sum()` function. From the below image we found that there are no null values present in our dataset. So we can skip handling the missing values step.

```
# Activity 1.1: Load and Explore Dataset
print("Loading Adult Census Income Dataset...")
df = pd.read_csv('adult.csv')

print(f"Dataset Shape: {df.shape}")
print("\nDataset Info:")
print(df.info())
```

```
Checking for missing values:
age          0
workclass    0
fnlwgt       0
education    0
education.num 0
marital.status 0
occupation   0
relationship 0
race         0
sex          0
capital.gain  0
capital.loss  0
hours.per.week 0
native.country 0
income       0
dtype: int64
```

Activity 2: Data Preparation

As we have understood how the data is, let's pre-process the collected data.

The download data set is not suitable for training the machine learning model as it might have so much randomness so we need to clean the dataset properly in order to fetch good results. This activity includes the following steps.

- **Handling missing values**
- **Handling Outliers**

Note: These are the general steps of pre-processing the data before using it for machine learning. Depending on the condition of your dataset, you may or may not have to go through all these steps.

Activity 2.1: Handling missing values

- For checking the null values, `df.isna().any()` function is used. To sum those null values we use `.sum()` function. From the below image we found that there are no null values present in our dataset. So we can skip handling the missing values step.

```

# Activity 2.1: Handle Missing Values
print("\nChecking for missing values:")
print(df.isnull().sum())

# Check for '?' values which represent missing data
print("\nChecking for '?' values:")
for col in df.columns:
    if df[col].dtype == 'object':
        print(f"{col}: {(df[col] == '?').sum()}")

# Replace '?' with 'Unknown' in workclass and occupation
df['workclass'] = df['workclass'].replace('?', 'Unknown')
df['occupation'] = df['occupation'].replace('?', 'Unknown')
df['native.country'] = df['native.country'].replace('?', 'Unknown')

```

```

Checking for '?' values:
workclass: 1836
education: 0
marital.status: 0
occupation: 1843
relationship: 0
race: 0
sex: 0
native.country: 583
income: 0

```

Activity 2.2: Handling Outliers

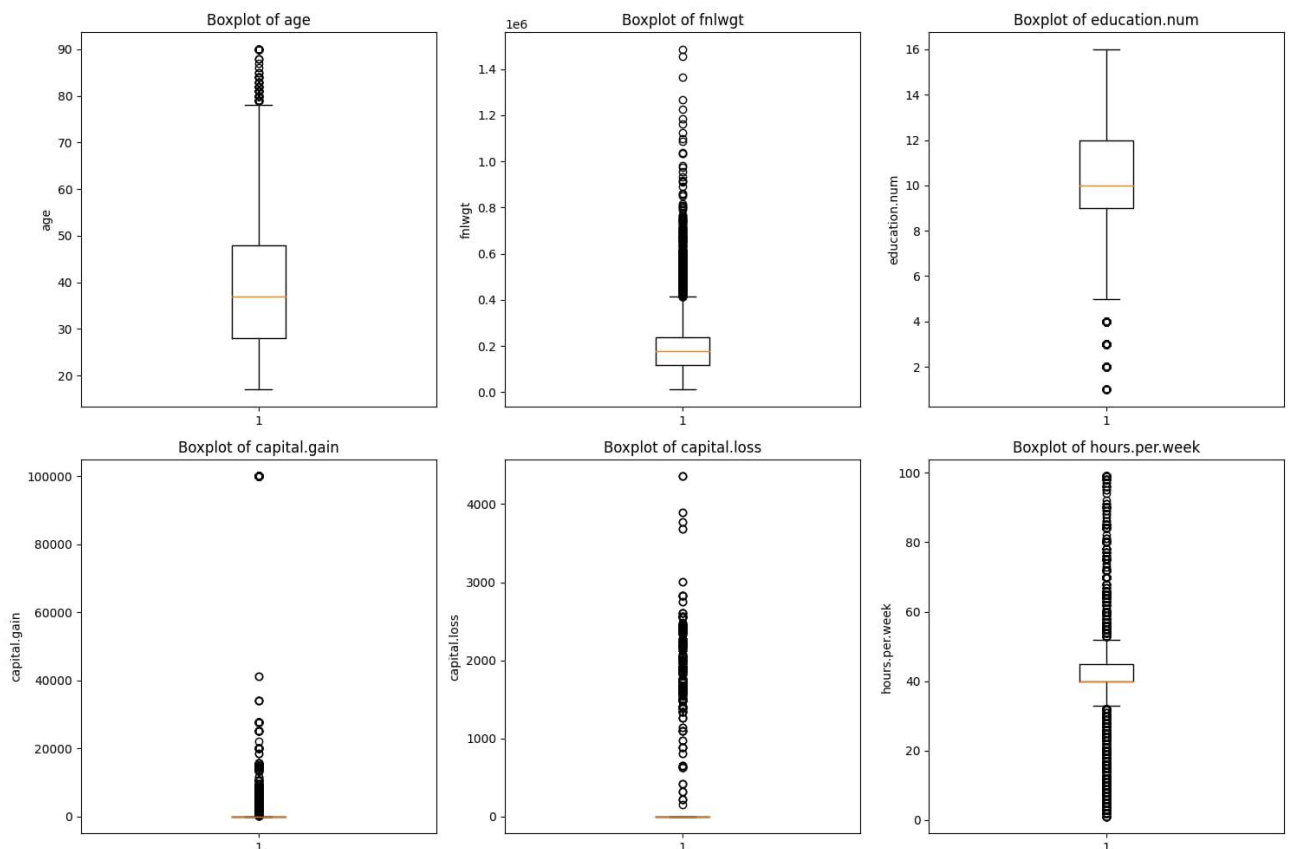
Outlier detection is crucial in financial prediction, such as the forecasting of adult income. Code in the attached picture checks numeric features like age, final weight, education level, capital gains and losses and hours per week for outliers (drawing boxplots). They also assist in finding values that are significantly further away from the normal range (possible outliers) that could cause problems with model estimation. Age-related outliers are to be expected and should not be removed, recognizing their relevance in income prediction. This approach ensures cleaner data without throwing out those useful variations which mirror what's happening in the real world.

```
# Activity 2.3: Outlier Detection and Treatment
plt.figure(figsize=(15, 10))
numeric_cols = ['age', 'fnlwgt', 'education.num', 'capital.gain', 'capital.loss', 'hours.per.week']

for i, col in enumerate(numeric_cols, 1):
    plt.subplot(2, 3, i)
    plt.boxplot(df[col])
    plt.title(f'Boxplot of {col}')
    plt.ylabel(col)

plt.tight_layout()
plt.savefig('outlier_analysis.png')
plt.show()

print("Outlier analysis completed. Age outliers are considered normal for adult income prediction.")
```



Milestone 3: Exploratory Data Analysis

Activity 1: Descriptive statistical

Descriptive analysis is to study the basic features of data with the statistical process. Here we used a pandas function called describe. With this describe function we can understand the unique, top and frequent values of categorical features. And we can find mean, std, min, max and percentile values of continuous features.

```
29 print("\nDataset Description:")
30 print(df.describe())
```

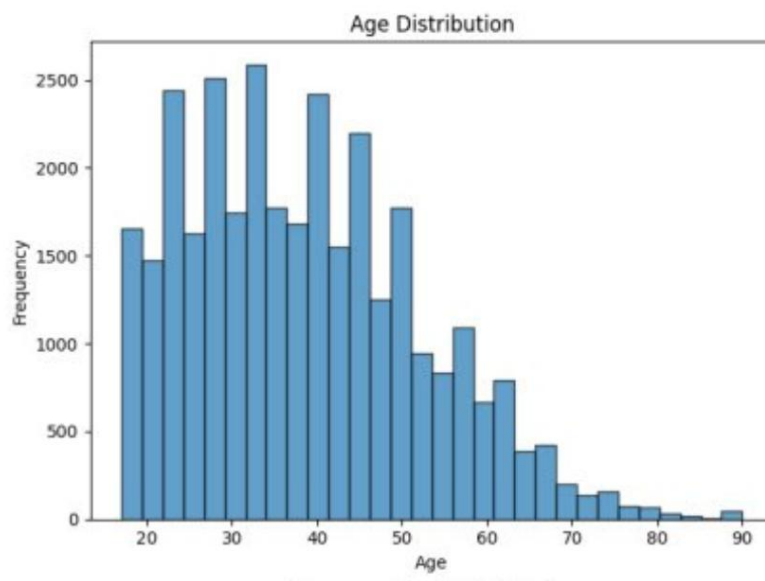
Dataset Description:						
	age	fnlwgt	education.num	capital.gain	capital.loss	\
count	32561.000000	3.256100e+04	32561.000000	32561.000000	32561.000000	
mean	38.581647	1.897784e+05	10.080679	1077.648844	87.303830	
std	13.640433	1.055500e+05	2.572720	7385.292085	402.960219	
min	17.000000	1.228500e+04	1.000000	0.000000	0.000000	
25%	28.000000	1.178270e+05	9.000000	0.000000	0.000000	
50%	37.000000	1.783560e+05	10.000000	0.000000	0.000000	
75%	48.000000	2.370510e+05	12.000000	0.000000	0.000000	
max	90.000000	1.484705e+06	16.000000	99999.000000	4356.000000	

Activity 2: Visual analysis

Visual analysis is the process of using visual representations, such as charts, plots, and graphs, to explore and understand data. It is a way to quickly identify patterns, trends, and outliers in the data, which can help to gain insights and make informed decisions.

Activity 2.1: Univariate analysis

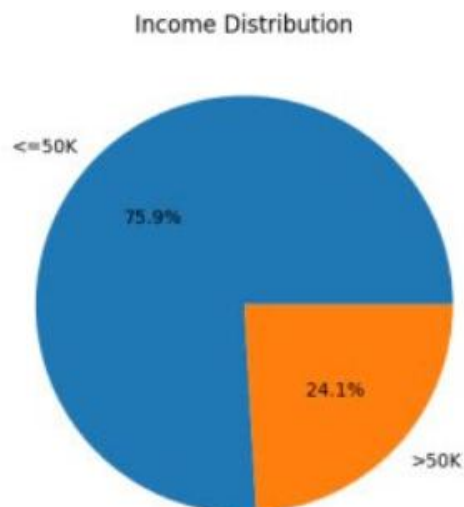
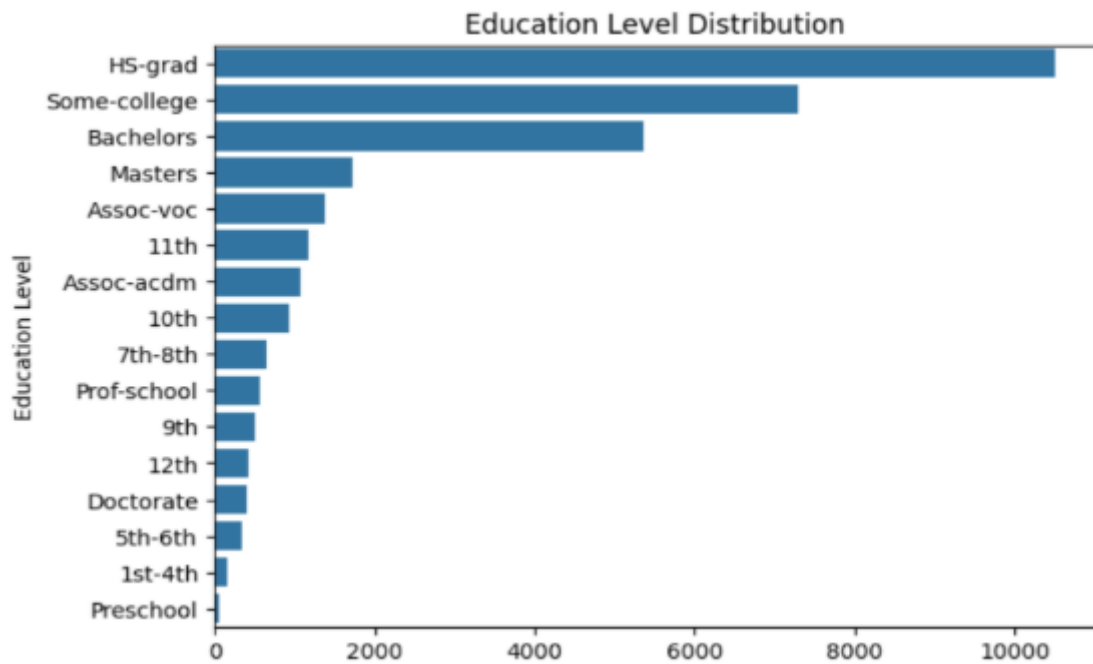
Univariate analysis is understanding the data with single feature. Here we have displayed a histogram, Pie chart, and Horizontal bar plot. Age Distribution ranges from 15 to 65years



The histogram above describes the distribution of Age in the dataset and a KDE plot is used to find skewness of the age and range of distribution of ages.

Seaborn package provides a wonderful function countplot. It is more useful for categorical features. With the help of countplot, we can find Number of unique values in the feature.

The chart below highlights that most individuals have completed high school, while very few have only reached preschool education.

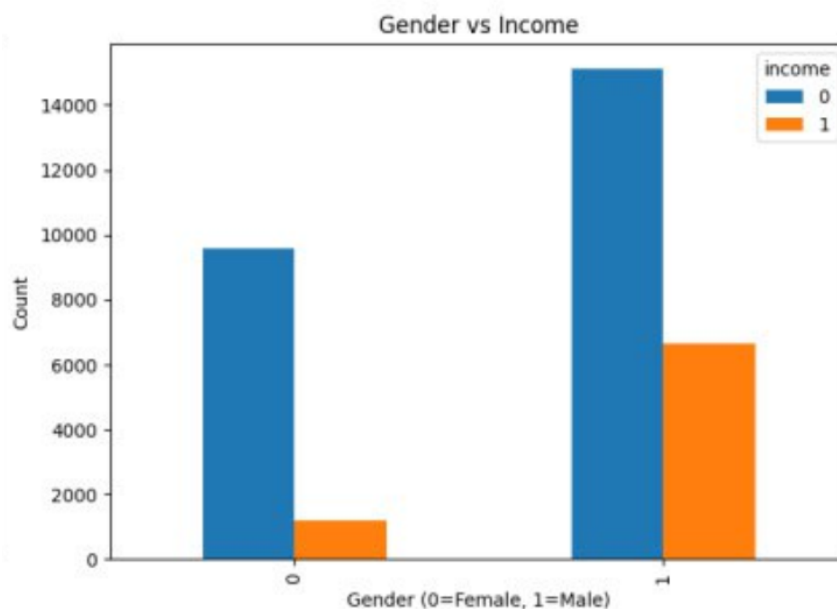


The **pie chart** above visually represents the proportion of a population within two income categories:

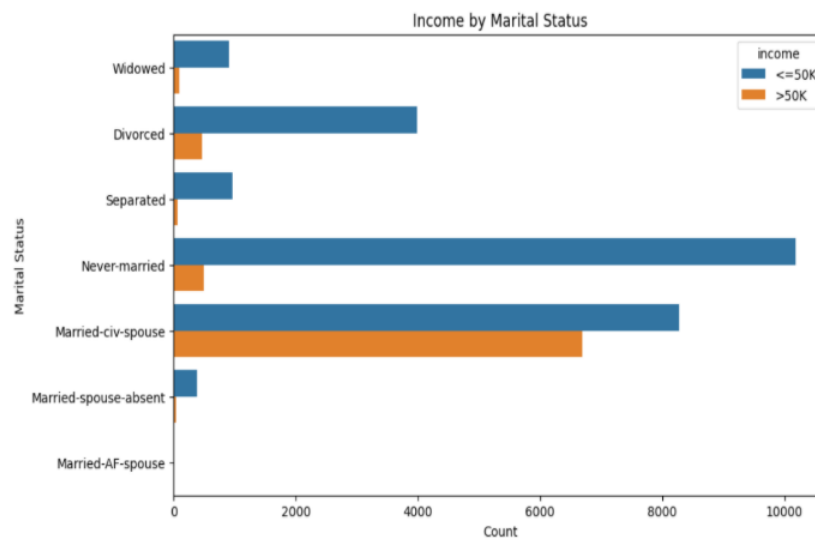
- **<= 50K (Less than or equal to 50,000)**: This segment is colored blue and represents **75.9%** of the total population.
- **> 50K (Greater than 50,000)**: This segment is colored orange and represents **24.1%** of the total population.

Activity 2.2: Bivariate analysis

Bivariate analysis is a statistical method that involves the analysis of two variables to determine the empirical relationship between them. Here we have grouped bar plots and horizontally grouped Bar charts.



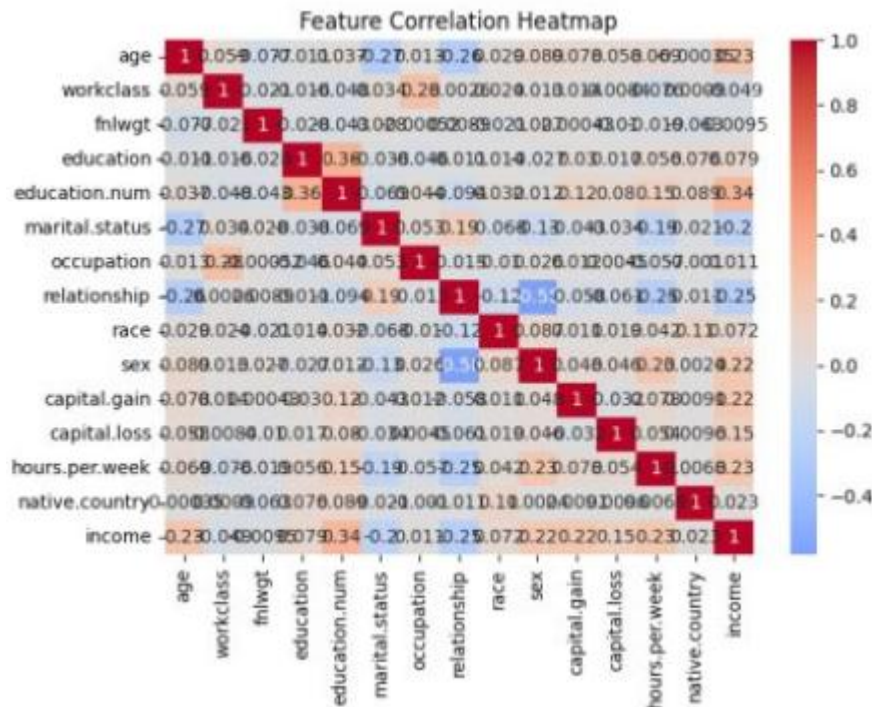
Males consistently outnumber females in both income groups, with the gap widening at higher income levels. There is an approximate difference of 4000 between males and females in earning $\leq 50k$, Males are Dominated in earning $>50k$.



The above chart shows that "Married-civ-spouse" individuals have the highest representation in both lower and higher income brackets compared to other marital statuses.

Activity 2.3: Multivariate analysis

In simple words, multivariate analysis means to find the relation between multiple features.



From the above heatmap we can clearly conclude that higher income is positively associated with greater education, capital gains, hours worked per week, and age.

- Each square in the heatmap shows the correlation between the feature on its row and the feature on its column.
- The color scale on the right indicates the strength and direction of the correlation, with warmer colors (reds) suggesting a stronger positive correlation (as one variable increases, the other tends to increase) and cooler colors (blues) indicating a stronger negative correlation (as one increases, the other tends to decrease).
- A value of 1.0 (brightest red) indicates a perfect positive correlation, while values close to 0 (lighter colors) suggest a weak or no linear correlation.
- This type of graph is useful for quickly identifying relationships and dependencies between variables in a dataset

Encoding the Categorical Features:

- The categorical Features are can't be passed directly to the Machine Learning Model. So we convert them into Numerical data based on their order. This Technique is called Encoding.
- Here we are importing Label Encoder from the Sklearn Library.
- Here we are applying fit_transform to transform the categorical features to numerical features.

```
# Activity 2.2: Handle Categorical Values
print("\nCategorical columns analysis:")
categorical_cols = ['workclass', 'education', 'marital.status', 'occupation',
                    'relationship', 'race', 'sex', 'native.country', 'income']

for col in categorical_cols:
    print(f"\n{col} value counts:")
    print(df[col].value_counts())

# Label Encoding
le = LabelEncoder()
for col in categorical_cols:
    df[col] = le.fit_transform(df[col])

print("\nAfter Label Encoding:")
print(df.head())
```

Splitting data into train and test

Now let's split the Dataset into train and test sets. First split the dataset into x and y and then split the data set.

Here x and y variables are created. On x variable, df is passed with dropping the target variable. And on y target variable is passed. For splitting training and testing data we are using train_test_split() function from sklearn. As parameters, we are passing x, y, test_size, random state.

```
# Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(
    X_balanced, y_balanced, test_size=0.2, random_state=100, stratify=y_balanced
)

print(f"Training set size: {X_train.shape}")
print(f"Test set size: {X_test.shape}")
```

Handling Imbalanced dataset

- Imbalanced data is a common problem in machine learning and data analysis, where the number of observations in one class is significantly higher or lower than the other class. Handling imbalanced data is important to ensure that the model is not biased towards the majority class and can accurately predict the minority class.
- Here we are using SMOTE Technique.

```
# Apply SMOTE for balancing
smote = SMOTE(random_state=42)
X_balanced, y_balanced = smote.fit_resample(X, y)

print("After SMOTE balancing:")
print(pd.Series(y_balanced).value_counts())
```

Scaling

- Scaling is a technique used to transform the values of a dataset to a similar scale to improve the performance of machine learning algorithms. Scaling is important because many machine learning algorithms are sensitive to the scale of the input features.
- Here we are using Standard Scaler.
- This scales the data to have a mean of 0 and a standard deviation of 1. The formula is given by:
$$X_{\text{scaled}} = (X - X_{\text{mean}}) / X_{\text{std}}$$

```
from sklearn.preprocessing import StandardScaler
std_scaler = StandardScaler()

X_train = std_scaler.fit_transform(X_train)
X_train = pd.DataFrame(X_train, columns=x.columns)

X_test = std_scaler.transform(X_test)
X_test = pd.DataFrame(X_test, columns=x.columns)
```

Milestone 4: Model Building

Activity 1: Training the model in multiple algorithms

Now our data is cleaned and it's time to build the model. We can train our data on different algorithms. For this project we are applying three classification algorithms. The best model is saved based on its performance.

Activity 1.1: Logistic Regression model

Logistic Regression is the baseline model where we import the Logistic Regression from the sklearn library and import the linear_model .Next, we assign the model to the variable model1 and make the data fit and processing the data with respect to the training data.

We make the predictions on the train and test data using predict function and assigning the predicted values to the y_pred_train and y_pred_test

```
# 1. Logistic Regression
print("\n1. Training Logistic Regression...")
model1 = LogisticRegression(random_state=42, max_iter=1000)
model1.fit(X_train, y_train)

y_pred_train1 = model1.predict(X_train)
y_pred_test1 = model1.predict(X_test)

train_acc1 = accuracy_score(y_train, y_pred_train1)
test_acc1 = accuracy_score(y_test, y_pred_test1)
cv_score1 = cross_val_score(model1, X_balanced, y_balanced, cv=5).mean()

models['Logistic Regression'] = model1
results['Logistic Regression'] = {
    'train_accuracy': train_acc1,
    'test_accuracy': test_acc1,
    'cv_score': cv_score1
}
```

Activity 1.2: Decision tree model

First Decision Tree is imported from sklearn Library . DecisionTreeClassifier algorithm is initialised and training data is passed to the model with the .fit() function. Test data is predicted with .predict() function and saved in a new variable. We can find the Train and Test accuracy by X_train and X_test.

We have also performed hyperparameter tuning to the decision tree model. Hyperparameter tuning is used to tune the model with best hyper parameters to obtain more accuracy and actual prediction.

```
# 2. Decision Tree Classifier
print("\n2. Training Decision Tree Classifier...")
model2 = DecisionTreeClassifier(criterion="gini", max_depth=5, splitter="best", random_state=42)
model2.fit(X_train, y_train)

y_pred_train2 = model2.predict(X_train)
y_pred_test2 = model2.predict(X_test)

train_acc2 = accuracy_score(y_train, y_pred_train2)
test_acc2 = accuracy_score(y_test, y_pred_test2)
cv_score2 = cross_val_score(model2, X_balanced, y_balanced, cv=5).mean()

models['Decision Tree'] = model2
results['Decision Tree'] = {
    'train_accuracy': train_acc2,
    'test_accuracy': test_acc2,
    'cv_score': cv_score2
}

print(f"Train Accuracy: {train_acc2:.4f}")
print(f"Test Accuracy: {test_acc2:.4f}")
print(f"Cross Validation Score: {cv_score2:.4f}")

# Calculate additional metrics for Decision Tree
precision2 = precision_score(y_test, y_pred_test2)
recall2 = recall_score(y_test, y_pred_test2)
f1_2 = f1_score(y_test, y_pred_test2)

print(f"Precision: {precision2:.4f}")
print(f"Recall: {recall2:.4f}")
print(f"F1 Score: {f1_2:.4f}")
print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred_test2))
```

Activity 1.3: Random forest model

Random Forest Classifier is an ensemble learning method that combines multiple decision trees to improve prediction accuracy and reduce overfitting. Here training data is passed to the model and assigned to the variable as model4 with .fit() function. Test data is predicted with model3.predict() function and saved in a new variable. For evaluating the model accuracy is calculated. For obtaining the best accuracy we use hyper parameter tuning to tune the model with the best hyper parameters using the Grid Search CV.

```
# 4. Random Forest Classifier
print("\n4. Training Random Forest Classifier...")
model3 = RandomForestClassifier(criterion="gini", max_depth=7, n_estimators=100, random_state=42)
model3.fit(X_train, y_train)

y_pred_train3 = model3.predict(X_train)
y_pred_test3 = model3.predict(X_test)

train_acc3 = accuracy_score(y_train, y_pred_train3)
test_acc3 = accuracy_score(y_test, y_pred_test3)
cv_score3 = cross_val_score(model3, X_balanced, y_balanced, cv=5).mean()

models['Random Forest'] = model3
results['Random Forest'] = {
    'train_accuracy': train_acc3,
    'test_accuracy': test_acc3,
    'cv_score': cv_score3
}
print(f"Train Accuracy: {train_acc3:.4f}")
print(f"Test Accuracy: {test_acc3:.4f}")
print(f"Cross Validation Score: {cv_score3:.4f}")
```

Activity 1.4: Adaboost Classifier

AdaBoost (Adaptive Boosting) Classifier is an ensemble learning technique that combines multiple weak learners—usually decision stumps (one-level decision trees)—to form a strong classifier. It works by focusing more on the data points that previous models misclassified, adjusting their weights in each iteration. The algorithm is initialized and training data is passed to the model and assigned to the variable as model4 with `.fit()` function. Test data is predicted with `model4.predict()` function and saved in a new variable. For evaluating the model accuracy is calculated.

```
# 5. AdaBoost Classifier
print("\n5. Training AdaBoost Classifier...")
model4 = AdaBoostClassifier(n_estimators=50, learning_rate=1.0, random_state=42)
model4.fit(X_train, y_train)

y_pred_train4 = model4.predict(X_train)
y_pred_test4 = model4.predict(X_test)

train_acc4 = accuracy_score(y_train, y_pred_train4)
test_acc4 = accuracy_score(y_test, y_pred_test4)
cv_score4 = cross_val_score(model4, X_balanced, y_balanced, cv=5).mean()

models['AdaBoost'] = model4
results['AdaBoost'] = {
    'train_accuracy': train_acc4,
    'test_accuracy': test_acc4,
    'cv_score': cv_score4
}

print(f"Train Accuracy: {train_acc4:.4f}")
print(f"Test Accuracy: {test_acc4:.4f}")
print(f"Cross Validation Score: {cv_score4:.4f}")
```

Milestone 5: Performance Testing & Hyperparameter Tuning

Activity 1: Testing model with multiple evaluation metrics

Multiple evaluation metrics means evaluating the model's performance on a test set using different performance measures. This can provide a more comprehensive understanding of the model's strengths and weaknesses. We are using evaluation metrics for classification tasks including accuracy, precision, recall, support and F1-score.

Activity 1.1: Compare the model

For comparing the above four models, comparison_df dataframe is used which will help us identify the best model based on the evaluation metrics.

```
# Model Comparison and Selection
print("\n=== MODEL COMPARISON ===")
comparison_df = pd.DataFrame(results).T
print(comparison_df)

# Select best model based on test accuracy and cross-validation score
best_model_name = comparison_df['test_accuracy'].idxmax()
best_model = models[best_model_name]

print(f"\nBest Model: {best_model_name}")
print(f"Test Accuracy: {comparison_df.loc[best_model_name, 'test_accuracy']:.4f}")
```

```
1. Training Logistic Regression...
Train Accuracy: 0.7643
Test Accuracy: 0.7640
Cross Validation Score: 0.7478

2. Training Decision Tree Classifier...
Train Accuracy: 0.8289
Test Accuracy: 0.8269
Cross Validation Score: 0.7984
Precision: 0.9042
Recall: 0.7312
F1 Score: 0.8085
Confusion Matrix:
[[4561 383]
 [1329 3615]]

3. Hyperparameter Tuning for Decision Tree...
Best parameters: {'criterion': 'gini', 'max_depth': 7, 'splitter': 'best'}
Best cross-validation score: 0.8453

4. Training Random Forest Classifier...
Train Accuracy: 0.8590
Test Accuracy: 0.8535
Cross Validation Score: 0.8278

5. Training AdaBoost Classifier...
Train Accuracy: 0.8430
Test Accuracy: 0.8421
Cross Validation Score: 0.8213

=== MODEL COMPARISON ===
              train_accuracy  test_accuracy  cv_score
Logistic Regression      0.764260      0.763956  0.747836
Decision Tree            0.828858      0.826861  0.798402
Random Forest            0.858996      0.853459  0.827832
AdaBoost                 0.843042      0.842132  0.821278

Best Model: Random Forest
Test Accuracy: 0.8535

Best model (Random Forest) saved as 'model.joblib'
```

Activity 2: Comparing model accuracy before & after applying hyperparameter tuning (The image above shows the results of training four machine learning models).

- **Random Forest** had the highest **test accuracy (85.35%)** so it is selected as the final model.
- **Decision Tree** gave the **best cross-validation score (84.33%)** after hyperparameter tuning.
- **AdaBoost** showed a good balance with over **84% accuracy** on both train and test data.
- **Logistic Regression** performed the least, with **~76% accuracy** on both train and test sets.
- Final model was saved as model.joblib based on overall performance

Milestone 6: Model Deployment

Activity 1: Save the best model

Saving the best model after comparing its performance using different evaluation metrics means selecting the model with the highest performance. This can be useful in avoiding the need to retrain the model every time it is needed and also to be able to use it in the future.

```
# Save the best model
joblib.dump(best_model, 'model.joblib')
print(f"\nBest model ({best_model_name}) saved as 'model.joblib'")
```

Activity 2: Integrate with Web Framework

In this section, we will be building a web application that is integrated to the model we built. A UI is provided for the users where he has to enter the values for predictions. The entered values are given to the saved model and prediction is showcased on the UI.

This section has the following tasks

- Building HTML Pages
- Building server-side script
- Run the web application

Activity 2.1: Building Html Page:

For this project create HTML file namely index.html, predict.html and result.html and save them in the templates folder.

Activity 2.2: Build Python code:

Import the libraries

```
# app.py
import numpy as np
import pandas as pd
import joblib
from flask import Flask, request, render_template
import warnings
warnings.filterwarnings('ignore')
```

Load the saved model. Importing the flask module in the project is mandatory. An object of Flask class is our WSGI application. Flask constructor takes the name of the current module (name) as argument.

```
# Load the trained model
try:
    model = joblib.load(r"C:\Users\Sanjoli\OneDrive\Desktop\Financial_Forecasting_Project\model.joblib")
    print("Model loaded successfully!")
except Exception as e:
    print(f"Error loading model: {e}")
    model = None
```

Render HTML page:

```
@app.route("/")
def home():
    """Render the home page."""
    return render_template('index.html')
```

Here we will be using a declared constructor to route to the HTML page which we have created earlier.

In the above example, '/' URL is bound with the index.html function. Hence, when the home page of the web server is opened in the browser, the html page will be rendered. Whenever you enter the values from the html page the values can be retrieved using POST Method.

Retrieves the value from UI:

Here we are routing our app to predict() function. This function retrieves all the values from the HTML page using Post request. That is stored in an array. This array is passed to the model.predict() function. This function returns the prediction. And this prediction value will be rendered to the text that we have mentioned in the submit.html page earlier.

```
def predict():
    """Render the prediction page."""
    return render_template('predict.html')

@app.route('/submit', methods=["POST"])
def submit():
    """Handle form submission and make predictions."""
```

```

@app.route('/predict')
def predict():
    """Render the prediction page."""
    return render_template('predict.html')

@app.route('/submit', methods=["POST"])
def submit():
    """Handle form submission and make predictions."""
    try:
        if model is None:
            return render_template("result.html",
                                   result="Model not loaded. Please check the model file.",
                                   prediction="Error")

        # Reading the inputs given by the user
        input_features = []

        # Get form data in the correct order
        age = int(request.form.get('age'))
        workclass = int(request.form.get('workclass'))
        fnlwgt = int(request.form.get('fnlwgt', 0))
        education = int(request.form.get('education'))
        education_num = int(request.form.get('education_num', 0))
        marital_status = int(request.form.get('marital_status'))
        occupation = int(request.form.get('occupation'))
        relationship = int(request.form.get('relationship'))
        race = int(request.form.get('race'))
        sex = int(request.form.get('sex'))
        capital_gain = int(request.form.get('capital_gain', 0))
        capital_loss = int(request.form.get('capital_loss', 0))
        hours_per_week = int(request.form.get('hours_per_week'))
        native_country = int(request.form.get('native_country'))

        # Create feature array
        input_features = [age, workclass, fnlwgt, education, education_num,
                           marital_status, occupation, relationship, race, sex,
                           capital_gain, capital_loss, hours_per_week, native_country]

        # Convert to numpy array and reshape for prediction
        input_array = np.array(input_features).reshape(1, -1)

        # Make prediction
        prediction = model.predict(input_array)[0]

        # Determine the result message based on the prediction value
        if prediction == 1:
            result = "Your income is predicted to be more than $50,000. Yes, you are ready for investment. Invest wisely!"
            prediction_text = ">50K"
        else:
            result = "Your income is predicted to be less than $50,000. Better to invest your money to learn skills."
            prediction_text = "<=50K"

        return render_template("result.html", result=result, prediction=prediction_text)

```

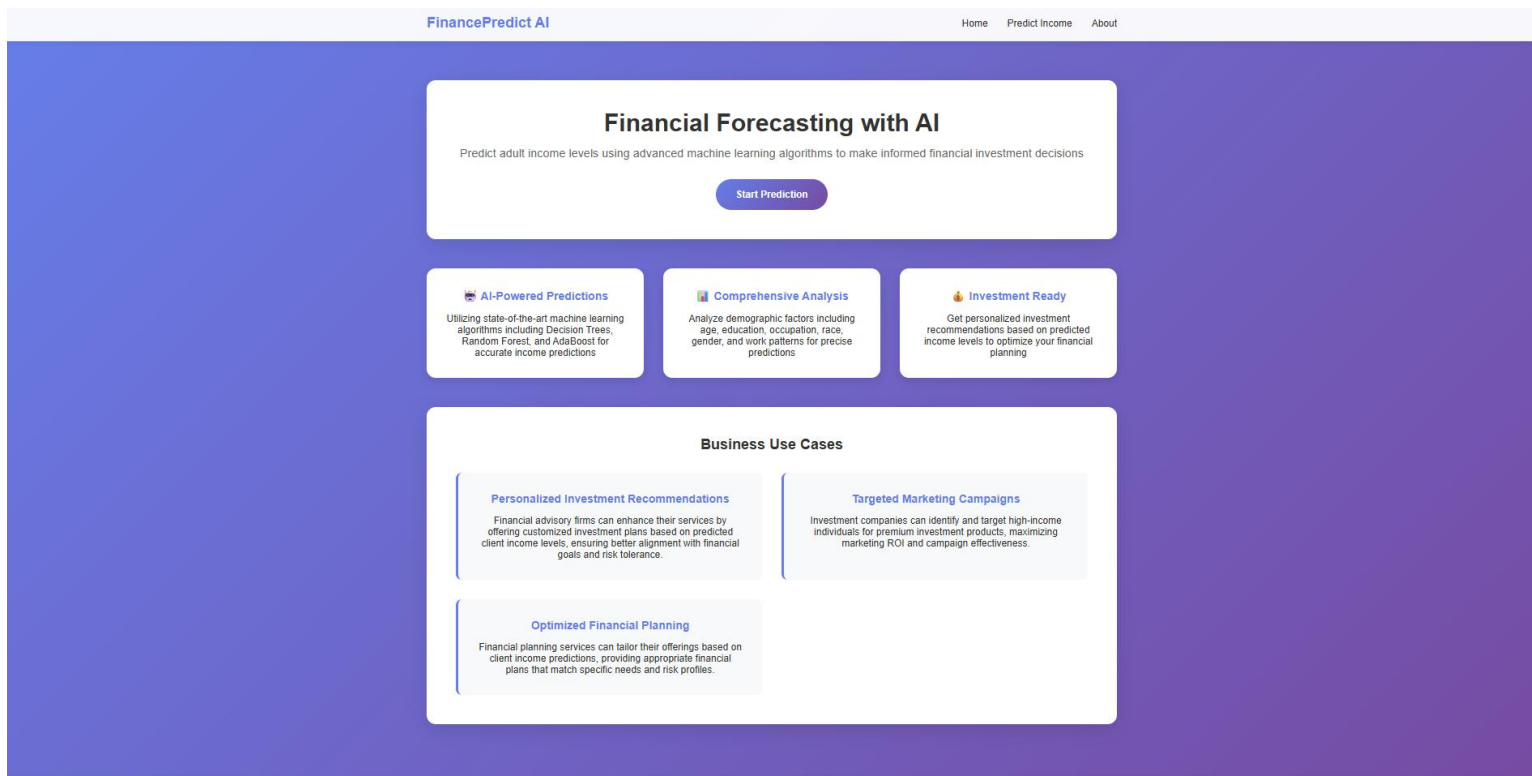
Main Function:

```
if __name__ == '__main__':  
    app.run(debug=True, port=4000, host='127.0.0.1')
```

Activity 2.3: Run the web application

- Open vs code application in the search menu.
- Navigate to the folder where your flask folder of your files exist.
- Click on the view button in the vs code nav bar and click on the terminal option in the dropdown menu.
- Now type “flask run” command
- You will have a link displayed in the terminal as “<http://127.0.0.1:4000>”.
- Double click on the link then you will be navigated to the web application.
- Click on the predict button in the nav bar , enter the inputs, click on the predict button, and see the result/prediction in the result.html..

Now. Go the web browser and write the localhost url (<http://127.0.0.1:5000>) to get the below result.



Income Prediction Form

127.0.0.1:5000/predict

Income Prediction Form

HomePredict IncomeAbout

Instructions: Please fill in all the required information below. The prediction model will analyze your demographic and occupational data to predict your income bracket and provide personalized financial investment recommendations.

Age:

45

Work Class:

Private

Final Weight:

100004

Education Level:

12th

Education Years:

10

Marital Status:

Married-civ-spouse

Occupation:

Armed-Forces

Relationship:

Select Relationship

Race:

Asian-Pac-Islander

Gender:

Female

Capital Gain:

2

Capital Loss:

0

Hours per Week:

34

Native Country:

Greece

Predict Income & Get Investment Advice

Prediction Results

127.0.0.1:5000/submit

Prediction Results

HomeStewie Griffin, [13 Jun 2025 11:58:14]Predict IncomeAbout

Income Prediction: >50K

Your income is predicted to be more than \$50,000. Yes, you are ready for investment. Invest wisely!

🔥 Investment Recommendations for High-Income Earners

- Consider diversified portfolio with stocks, bonds, and real estate
- Explore tax-advantaged accounts like 401(k) and IRA
- Look into high-yield investment opportunities
- Consider professional financial advisory services
- Evaluate international investment opportunities

Make Another Prediction

Back to Home

www.smartinternz.com

Page 23 of 22

Milestone 7: Project Demonstration & Documentation

Below are the mentioned deliverables to be submitted along with other deliverables

Activity 1:- Record explanation Video for project end to end solution

<https://drive.google.com/file/d/18ISvTLIUNS-xAotVGMIgTwogklf5wuq/view?usp=drivesdk>

Activity 2:- Project Documentation-Step by step project development procedure

Create document as per the template provided